

Desafio Elite Dev

A proposta deste teste é validar seus conhecimentos técnicos em desenvolvimento Front-End e Back-End, lógica de programação, e sua capacidade de entender e atender a demanda proposta.

Proposta de Solução

Você deve criar uma **Plataforma de Eventos e Ingressos**, onde um organizador publica eventos e um cliente compra ingressos.

O **organizador** monta um evento a partir de um catálogo de shows ou filmes vindo de uma API externa, definindo data, local, capacidade e preço. O **cliente** navega pelos eventos publicados, reserva seu lugar, paga de forma simulada, recebe um ingresso com código em QR e pode compartilhá-lo por link. Na entrada do evento, a **portaria** valida o ingresso.

O que queremos ver

Vivemos na era da IA, e sabemos o que isso significa aqui: qualquer enunciado colado numa ferramenta devolve um sistema inteiro. Um desenvolvedor nosso fez exatamente isso com este PDF, sem escrever mais nada, e recebeu a aplicação pronta.

Por isso **o escopo aqui é pequeno de propósito**. O que nos interessa não é o volume entregue: é **como você pensa**. As decisões que tomou, o que descartou pelo caminho, por que a tela é assim e não de outro jeito.

Fuja do **AI slop**: aquela interface que sai pronta da ferramenta e que você reconhece de longe, porque todo projeto gerado tem exatamente a mesma cara. O problema não é a IA ter feito, é ninguém ter escolhido nada.

Queremos ver a sua mão no resultado, e um sistema é o meio que escolhemos para você mostrar isso.

Requisitos Funcionais

Front-End:

- Navegação e busca pelos eventos publicados (shows ou filmes em cartaz), com data, local e preço.
- Criação e gerenciamento dos eventos pelo organizador.
- Fluxo de reserva, com seleção do lugar num mapa de assentos (cinema, teatro) ou da quantidade de ingressos (pista). Implemente um dos dois, ou os dois.
- Pagamento simulado, contemplando a confirmação e também a recusa.
- Área de "Meus ingressos", exibindo o ingresso e o seu código em QR.
- Tela de **portaria**, para validar o ingresso na entrada do evento, com retorno claro: válido, inválido, já utilizado ou evento errado.
- Leitura do QR **pela câmera** na portaria, tendo a digitação manual do código como alternativa.

Back-End:

- Gestão das chamadas para a API externa: **Ticketmaster Discovery** ou **TMDb**. Você pode usar uma, a outra, ou as duas.
developer.ticketmaster.com/products-and-docs/apis/discovery-api/v2
developer.themoviedb.org/docs
- Autenticação com três papéis distintos: **Organizador**, que cria e gerencia eventos, **Cliente**, que reserva, paga e recebe ingressos, e **Portaria**, que valida ingressos na entrada.
- Armazenamento dos eventos, das reservas e dos ingressos.
- Garantia de que o mesmo lugar não seja vendido duas vezes.
- Geração do ingresso com um código em QR que não possa ser forjado.
- Implementação de lógica para permitir que o cliente compartilhe um ingresso via um link gerado pela aplicação.
- Validação do ingresso na portaria, garantindo que o mesmo ingresso não seja validado duas vezes.

A cobrança deve ser simulada, sem transação financeira real. Se preferir, você pode usar o ambiente de testes de um provedor de pagamento de verdade.

Tecnologias Obrigatórias

- ✓ **Front-End:** React, com ou sem framework: Next.js, Vite, Remix ou o que você preferir.
- ✓ **Back-End:** NodeJS, Python ou Java. Fique à vontade com o framework: NestJS, Express, FastAPI, Django ou Spring Boot.
- ✓ **Banco de Dados:** Utilize qualquer distribuição. Certifique-se de incluir no README instruções claras sobre como configurar e utilizar o banco de dados escolhido.

Referências

Para ver como esses fluxos costumam ser resolvidos. Não copie; use como ponto de partida.

- **ingresso.com:** mapa de assentos de cinema.
- **eventim.com.br:** pista e setores por quantidade.
- **symppla.com.br:** criação de evento e checkout.

Requisitos Não Funcionais

Prazo: Você tem 7 dias corridos a partir do recebimento deste desafio para completá-lo.

Documentação: O projeto deve incluir um README detalhado explicando o passo a passo para configurar e executar a aplicação. Caso algo não esteja funcionando conforme o esperado, isso deve ser mencionado no README. A ausência de explicações impactará negativamente na nota final.

Dados de teste: Deixe semeados um organizador, dois clientes, um usuário de portaria e ao menos um evento publicado com ingressos disponíveis, para que possamos percorrer o fluxo sem montar tudo do zero.

Deploy: Não é obrigatório, mas facilita muito a avaliação do seu projeto: conseguimos ver funcionando antes de ler o código, e isso muda a leitura inteira a seu favor. Publicar a aplicação na Vercel ou em uma plataforma similar renderá um acréscimo de 1 ponto na sua nota final.

Opcionais

Nenhum dos itens abaixo é obrigatório, mas todos são considerados na avaliação.

- Busca e filtro de eventos, painel do organizador, cancelamento com devolução ao estoque.
- Mapa de assentos em tempo real, Docker Compose, testes, aplicação publicada.

Não precisa fazer: nota fiscal, revenda entre usuários, aplicativo nativo, recuperação de senha ou envio de ingresso por e-mail.

Uso de IA

Recomendamos usar IA. Usar bem é uma habilidade que valorizamos, e não tira ponto nenhum.

Conte quais ferramentas você usou, em que partes do projeto, e o que fez sem IA. Pode ser uma seção no README, um arquivo dedicado, ou o formato que preferir.

Isso não é cobrança, é oportunidade. Muita decisão que parece estranha numa leitura rápida se justifica quando você conta como chegou nela. Explicar o seu processo defende escolhas que, sem contexto, poderiam ser mal interpretadas por quem avalia.

Se você produziu artefatos no caminho, como specs, PRD, fluxo BMAD ou arquivos de contexto, **versione junto no repositório.** Ver como você conduziu a ferramenta conta a seu favor.

Entrega

Repositório GitHub: O código deve ser versionado e publicado em um repositório público no GitHub. Faça commits ao longo da semana, com mensagens descritivas, pois o histórico mostra o seu processo.

Envio: Ao finalizar o desafio, envie o link do repositório pelo formulário elitedev.verzel.com.br, indicando onde o código foi publicado e como executá-lo.

Dica

Faça o básico rodar de ponta a ponta e só depois agregue valor. Preferimos o fluxo inteiro simples e completo a um pedaço sofisticado com telas pela metade.

Fora isso, todo cuidado extra é bem-vindo! Recursos como **uma interface bem feita e agradável de usar**, documentação clara, organização do código, tratamento de erros, boas práticas de versionamento e testes básicos serão vistos como diferenciais. Mais do que as tecnologias em si, o que conta é como você estrutura a solução e explica suas decisões.

Adoramos iniciativa. Criatividade e dedicação serão bem avaliadas. Se você olhar a proposta e pensar "isso ficaria melhor com tal coisa", faça e conte no README por quê.

Faça seu projeto com carinho, que a gente vai avaliar com carinho.

Então dê o seu melhor e nos mostre o que VOCÊ tem de melhor. Boa sorte!